

Welcome to Week 2: Training Neural Networks

Last week's focus was on **inference**, which is the process of using an already-trained neural network to make predictions.

This week's focus is on **training**, which is the process of learning the optimal parameters (weights and biases) for a neural network based on a given training set.

The 3-Step Training Process in TensorFlow

We will continue using the handwritten digit recognition example (a 3-layer network: 25 units, 15 units, 1 unit). Training this model in TensorFlow involves three key steps:

1. Define the Model Architecture

- This step is identical to last week. You use `Sequential` to define the stack of layers.
- This tells TensorFlow how to perform the forward propagation (inference) calculation.

```
1 model = Sequential([
2     Dense(units=25, activation='sigmoid'),
3     Dense(units=15, activation='sigmoid'),
4     Dense(units=1, activation='sigmoid')
5 ])
```

2. Compile the Model

- This step configures the model for training.
- The most critical component you specify here is the **loss function** (or cost function) that you want the model to minimize.
- For a binary (0/1) classification problem like this, the standard loss function is **Binary Crossentropy**.

```
1 model.compile(loss='binary_crossentropy')
```

3. Train the Model ("Fit")

- This step actually runs the training process (i.e., gradient descent).
- You call `model.fit()` and provide your training data (X) and your target labels (Y).
- You also specify the number of **epochs**. An **epoch** is a technical term for one full pass or iteration of the learning algorithm over the entire training set.

```
1 model.fit(X, Y, epochs=100)
```

Why Understand "Under the Hood"?

While these three function calls are all you need to train a model, it's crucial to understand *what* they are doing. This conceptual framework is essential for **debugging** your model when it doesn't work as expected.

Training a Neural Network: The 3-Step Process 🧠

Training a neural network in TensorFlow follows the same three fundamental steps we used for training a logistic regression model. The TensorFlow functions `model.compile()` and `model.fit()` are powerful abstractions that handle these steps for us.

The 3-Step Framework (Recap from Logistic Regression)

1. **Specify the Model:** Define the function for forward propagation, $f(x)$, which computes the output based on inputs (x) and parameters (w, b).
 2. **Specify the Loss & Cost Functions:**
 - **Loss Function** $L(f(x), y)$: Measures the error on a *single* training example.
 - **Cost Function** $J(w, b)$: The *average* of the loss over the entire training set.
 3. **Minimize the Cost Function:** Use an optimization algorithm (like **gradient descent**) to find the parameters (w, b) that minimize J .
-

Mapping TensorFlow to the 3 Steps ⚙️

Here is how the TensorFlow code maps to this 3-step framework.

Step 1: Specify the Model

This is accomplished by defining the network's architecture using `Sequential`.

- **Code:**

```
1 model = Sequential([
2     Dense(units=25, activation='sigmoid'),
3     Dense(units=15, activation='sigmoid'),
4     Dense(units=1, activation='sigmoid')
5 ])
```

- **What it does:** This code defines the complex forward propagation function $f(x)$. It tells TensorFlow all the layers, their connections, and the parameters (all the $\vec{w}^{[l]}$ and $b^{[l]}$ for all layers) that are needed to compute a prediction.
-

Step 2: Specify the Loss and Cost Function

This is accomplished by calling `model.compile()`.

- **Code:**

```
1 model.compile(loss='binary_crossentropy')
```

- **What it does:** This tells TensorFlow what to use for the **loss function**.
 - **Binary Crossentropy:** For binary (0/1) classification, we use this loss. It is the *exact same formula* we used for logistic regression:
$$L(f(x), y) = -y \log(f(x)) - (1 - y) \log(1 - f(x))$$
 - TensorFlow automatically defines the **cost function** $J(W, B)$ (where W, B represent *all* parameters in the network) as the average of this loss over all training examples.
 - **For Other Problems:** If this were a regression (predicting a number) problem, you would specify a different loss, such as 'mean_squared_error'.
-

Step 3: Minimize the Cost Function

This is accomplished by calling `model.fit()`.

- **Code:**

```
1 | model.fit(X, Y, epochs=100)
```

- **What it does:** This single function call runs the entire optimization process.
 1. It repeatedly uses **gradient descent** (or a more advanced optimizer) to update all the parameters (W, B) in the network.
 2. To compute the necessary derivatives ($\frac{\partial J}{\partial w}$ and $\frac{\partial J}{\partial b}$), it uses a crucial algorithm called **backpropagation**.
 3. **Epochs:** This parameter tells TensorFlow how many times to iterate over the entire training dataset. An `epochs=100` means it will perform 100 passes of gradient descent.
-

The Role of Libraries (Abstraction)

Libraries like TensorFlow and PyTorch are now mature enough to handle the most complex parts of training (like backpropagation and optimization) for you. This is similar to calling a built-in `sort()` function instead of writing a sorting algorithm from scratch. While you will use these libraries, understanding the 3 steps they are performing "under the hood" is essential for debugging and building effective models.